

Prim's Algorithm & Advanced Graph Transformations

Algorithm Design & Analysis Tutorial

August 18, 2026

Conceptual Comparison: Kruskal's vs. Prim's

Both algorithms find the Minimum Spanning Tree using greedy strategies, but their growth patterns are entirely different.

Kruskal's Algorithm: The Global Strategy

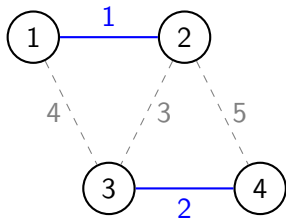
- ▶ **Behavior:** Looks at the entire graph globally, picking the absolute cheapest edges anywhere.
- ▶ **Structure:** Builds multiple disconnected components (a forest) that eventually merge together.

Prim's Algorithm: The Local Strategy

- ▶ **Behavior:** Starts at one specific, localized point and only evaluates edges at the immediate boundary of its current territory.
- ▶ **Structure:** Grows a *single, continuous, connected component* outward until all vertices are included.

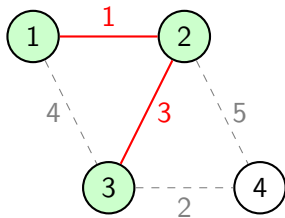
Pictorial Visualization: Kruskal's vs. Prim's

Kruskal's Algorithm



Edges added globally based purely on weight.
Forms a disconnected forest initially.

Prim's Algorithm



Edges added locally from start node (1).
Forms a single growing connected component.

Prim's Algorithm: The Strategy

The 4 Steps:

1. Pick ANY node to start. Mark it as "visited".
2. Evaluate all edges connecting visited nodes to unvisited nodes.
3. Greedily select the edge with the absolute minimum weight.
4. Add the new node to your network, mark it as "visited", and repeat step 2 until all vertices are connected.

The Efficiency Solution: Priority Queue (Min-Heap)

To quickly find the cheapest available edge without scanning all edges repeatedly ($\mathcal{O}(V^2)$ time), we utilize a Min-Heap. The priority queue dynamically maintains the cheapest crossing edges at the top, allowing extraction in $\mathcal{O}(\log V)$ time.

C++ Implementation: Prim's Algorithm

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4
5  using namespace std;
6  typedef pair<int, int> pii;
7
8  int prims(int V, vector<vector<pii>>& adj) {
9      priority_queue<pii, vector<pii>, greater<pii>> pq;
10     vector<bool> visited(V, false);
11     int total_cost = 0;
12
13     pq.push({0, 0}); // Start from node 0 with weight
14                       0
15
16     while (!pq.empty()) {
17         int weight = pq.top().first;
18         int u = pq.top().second;
19         pq.pop();
20
21         if (visited[u]) continue;
```

Prim's Algorithm: Complexity Analysis

Let V be the number of vertices and E be the number of edges.

Time Complexity: $\mathcal{O}(E \log V)$

- ▶ Initializing the graph and priority queue takes $\mathcal{O}(V)$.
- ▶ Each vertex is extracted from the min-heap at most once. V extractions take $\mathcal{O}(V \log V)$.
- ▶ For each extracted vertex, we push its adjacent edges. In total, we push at most E edges into the min-heap, taking $\mathcal{O}(E \log V)$.
- ▶ Total Time: $\mathcal{O}((V + E) \log V)$, which simplifies to $\mathcal{O}(E \log V)$ for connected graphs.

Space Complexity: $\mathcal{O}(V + E)$

- ▶ Adjacency list representation requires $\mathcal{O}(V + E)$.
- ▶ The priority queue (min-heap) and visited array take $\mathcal{O}(V)$ auxiliary space.

Problem 1: The Water Supply Network

Problem Statement:

A village has N houses. You need to ensure every house has access to water. For each house $i \in \{1, \dots, N\}$, you have two choices:

1. Build a well directly at house i for a cost W_i .
2. Lay a pipe between house i and house j for a cost $P_{i,j}$.

Goal: Find the absolute minimum total cost to ensure every house has water (either from its own well or via pipes connected to another house with a well).

The Graph Transformation

To solve this cleanly, we transform the problem into a standard Minimum Spanning Tree (MST) problem over an augmented graph.

Construction of Graph $G' = (V', E')$:

- ▶ Let $V = \{1, 2, \dots, N\}$ be the set of houses.
- ▶ Create a dummy node 0 (representing the subterranean water table).
- ▶ Define $V' = V \cup \{0\}$.
- ▶ Define the edge set E' such that:
 - ▶ For every potential pipe between i and j , add an edge (i, j) with weight $P_{i,j}$.
 - ▶ For every potential well at house i , add an edge $(0, i)$ with weight W_i .

Algorithm: Run Prim's Algorithm starting from Node 0 on G' .

Mathematical Proof of Correctness (Part 1)

Claim: The cost of the MST on G' equals the optimal cost of the water supply network.

Proof (By equivalence of valid subgraphs):

Let C be a valid water distribution configuration with minimal cost. C partitions the N houses into k disjoint sets, where each set forms a connected component via pipes, and exactly one house in each component has a well.

If a component had zero wells, it would lack water (invalid). If it had > 1 wells, we could remove one well (or a pipe) to strictly decrease the cost without losing connectivity, contradicting the assumption that C is minimal. Thus, the pipes in each component form a tree.

Mathematical Proof of Correctness (Part 2)

Now map configuration C to our graph G' :

- ▶ The pipes in C correspond to edges between nodes $\{1 \dots N\}$.
- ▶ The wells in C correspond to edges connected to node 0.

Since C consists of k disjoint tree components, and exactly one node in each component is connected to node 0, the union of these components and the edges to node 0 forms a single, connected, acyclic subgraph spanning all $N + 1$ vertices in G' . By definition, this subgraph is a spanning tree of G' . Because there is a direct bijection between valid minimal water configurations and spanning trees of G' , finding the spanning tree of G' with the absolute minimum total edge weight (the MST) guarantees finding the optimal water network. ■

C++ Solution: Water Supply Network (Part 1)

```
1  int minWaterCost(int n, vector<int>& wells, vector<
    vector<int>>& pipes) {
2      // Graph with n + 1 nodes (0 to n)
3      vector<vector<pii>> adj(n + 1);
4
5      // 1. Add edges from dummy node 0 to all houses (
        wells)
6      for (int i = 1; i <= n; i++) {
7          adj[0].push_back({wells[i-1], i});
8          adj[i].push_back({wells[i-1], 0});
9      }
10
11     // 2. Add regular pipe edges between houses
12     for (auto& pipe : pipes) {
13         int u = pipe[0], v = pipe[1], cost = pipe[2];
14         adj[u].push_back({cost, v});
15         adj[v].push_back({cost, u});
16     }
```

C++ Solution: Water Supply Network (Part 2)

```
1 // 3. Run Prim's Algorithm starting from Dummy
   Node 0
2 priority_queue<pii, vector<pii>, greater<pii>> pq;
3 vector<bool> visited(n + 1, false);
4 int total_cost = 0;
5
6 pq.push({0, 0});
7
8 while (!pq.empty()) {
9     int weight = pq.top().first;
10    int u = pq.top().second;
11    pq.pop();
12
13    if (visited[u]) continue;
14
15    visited[u] = true;
16    total_cost += weight;
17
18    for (auto edge : adj[u]) {
19        if (!visited[edge.second]) pq.push(edge);
20    }
21 }
```

Water Supply Network: Complexity Analysis

Let N be the number of houses and M be the number of given pipes. Our transformed graph G' has $V' = N + 1$ vertices and $E' = N + M$ edges.

Time Complexity: $\mathcal{O}((N + M) \log N)$

- ▶ Building the transformed graph G' takes $\mathcal{O}(N + M)$ time.
- ▶ Running Prim's algorithm on G' takes $\mathcal{O}(E' \log V')$.

Substituting our structural values, the bound tightly reduces to $\mathcal{O}((N + M) \log(N + 1))$, which is bounded by $\mathcal{O}((N + M) \log N)$.

Space Complexity: $\mathcal{O}(N + M)$

- ▶ The adjacency list for G' demands an $\mathcal{O}(N + M)$ memory footprint.
- ▶ The min-heap priority queue and visited tracker consume an additional $\mathcal{O}(N)$ memory.

Problem 2: The VIP Network & The Max Array

Problem Statement:

You are given an array A of size N containing distinct integers. You also have M "VIP" swap pairs that allow you to swap specific elements an infinite number of times for **zero cost**. However, you can also buy the ability to swap any other adjacent elements i and j for a cost $C_{i,j}$.

The Challenge:

1. **The Free Maximum:** Using *only* the zero-cost VIP swaps, what is the lexicographically largest array you can form?
2. **The Universal Upgrade:** What is the absolute minimum total cost to buy additional swaps so that you can arrange the array into *any* possible permutation?

Solution 2, Part 1: The Free Maximum (DSU)

If you can swap index 1 with 2, and 2 with 3, you can effectively move the element at index 1 to index 3.

The Algorithm:

- ▶ Initialize a Disjoint Set Union (DSU) for the N indices.
- ▶ For every zero-cost VIP swap (u, v) , perform `unite(u, v)`.
- ▶ Group the indices by their DSU root (their "team").
- ▶ For each connected group, extract the values at those indices, sort them in **descending order**, and place them back into the indices (which should be sorted in ascending order).

This mathematically guarantees the lexicographically largest possible array using only free operations!

Solution 2, Part 2: The Universal Upgrade (MST)

How do we guarantee we can form *any* permutation? The graph of indices needs to be a single, fully connected component.

The Algorithm:

- ▶ We don't start from scratch! The DSU from Part 1 has already grouped our nodes into connected "super-nodes" for zero cost.
- ▶ We treat each DSU root as a "super-node".
- ▶ We run Kruskal's or Prim's Algorithm on the remaining available paid swaps $C_{i,j}$.
- ▶ We only add a paid swap to our total cost if it connects two *different* DSU components (i.e., $\text{find}(u) \neq \text{find}(v)$).

Problem 3: The Complete Graph Trap (XOR MST)

Problem Statement:

You are given an array A of N integers (where $N \leq 10^5$). You are allowed to lay a cable between *any* two elements i and j . The cost of the cable is defined by the bitwise operation:

$$\text{Cost} = A[i] \oplus A[j]$$

Goal: Find the Minimum Spanning Tree of this network.

The Trap: Because every element can connect to every other element, the graph is a complete graph with $\mathcal{O}(N^2)$ edges (roughly 10 billion edges!). Running standard Prim's or Kruskal's will instantly result in a Time Limit Exceeded (TLE).

Problem 3: Hints

Hint 1: You cannot afford to calculate all $\mathcal{O}(N^2)$ edge weights and push them into a Min-Heap or sort them. You must find the minimum edges *without* explicitly building the graph.

Hint 2: To quickly find the minimum XOR pair between an element X and a large set of other elements, what data structure groups numbers by their binary prefixes?

Hint 3: Instead of a Priority Queue, can you maintain the available bits in a **Trie** (Prefix Tree) and modify your MST algorithm to query the Trie for the cheapest connections?